Git and GitHub for Physics
One Shared Guide for Every Operating System

UCC Physics Society

October 18, 2026

Contents

1 Git is not GitHub 2

2 Identity and authentication 2

3 Mental model and inspection commands 2

4 Local practice lab 2

5 Ignore generated and private files 3

6 Publish your own project 3

7 Branches and pull requests 4

8 Merge conflict practice 4

9 Safe recovery 4

10 Troubleshooting and privacy 5

11 Further reading 5

1 Git is not GitHub

Git records project history locally. GitHub hosts repositories online and adds pull requests and reviews. Saving, committing, and pushing are different operations; local Git works offline. The commands here work in PowerShell, Bash, and the VS Code integrated terminal. For a command reference, see the official Git documentation and Pro Git.

2 Identity and authentication

Check `git -version`, then configure the identity used on new commits: If Git is missing, follow the installation instructions in your operating-system guide or use the official Git download page.

```
git config --global user.name "Your Name"
git config --global user.email "YOUR_CHOSEN_EMAIL"
```

Replace both placeholders. If you want email privacy, obtain your GitHub no-reply address from account settings *before* making public commits. For one repository, omit `-global` and run inside it. Never alter another user's global configuration on a shared login.

Author identity is **not login**:

HTTPS: Use an approved credential manager/browser sign-in, or a narrowly scoped token when required. An account password is not a Git HTTPS password.

SSH: Follow GitHub's SSH guide. Add only your public key to GitHub; keep the private key private.

Do not put tokens in remote URLs, notebooks, screenshots, or source. Do not disable TLS or host-key checks to bypass authentication problems.

3 Mental model and inspection commands

Working tree $\xrightarrow{\texttt{git add}}$ staging area $\xrightarrow{\texttt{git commit}}$ local history

Pushing sends local commits to a remote repository; it does not automatically include uncommitted edits.

Command	Meaning
`git status`	What changed, what is staged, current branch
`git diff`	Unstaged changes
`git diff -staged`	Exact changes proposed for the next commit
`git log -oneline -5`	Recent commits
`git remote -v`	Remote names and URLs; check for credentials before sharing

Commits are deliberate snapshots, not automatic backups of every edit.

4 Local practice lab

Work in a new writable directory **outside the workshop repository**:

```
mkdir physics-git-practice
cd physics-git-practice
git init -b main
```

If an older Git does not support `-b`, use `git init` followed by `git branch -M main`. Create `README.md` in your editor:

```
# Pendulum lab
We will measure the period of a pendulum.
```

Then run:

```
git status
git add README.md
git diff --staged
git commit -m "Describe the pendulum experiment"
```

Add a second sentence about recording units, then:

```
git diff
git add README.md
git diff --staged
git commit -m "Record the measurement units"
git log --oneline -5
git status
```

Success: two commits, a clean working tree, and an explanation of the second change. If identity is missing, configure it as above and retry the commit.

5 Ignore generated and private files

Create `.gitignore` in your *practice repository*:

```
.venv/
__pycache__/
.ipynb_checkpoints/
.env
*.aux
*.log
*.out
*.toc
*.fls
*.fdb_latexmk
*.synctex.gz
*.blg
```

Do not blindly ignore all PDFs or CSVs. Selected release PDFs and small public datasets can be intentional deliverables. This workshop keeps deliverable PDFs but excludes temporary LaTeX build state.

Ignoring affects untracked files only: it does not remove tracked files or erase history. Do not treat it as secret protection. Stage named files rather than immediately running `git add .`; review `git status` and `git diff -staged` for private data, large output, and unrelated changes.

6 Publish your own project

Create an **empty** GitHub repository for this route: do not initialise it with a README, licence, or ignore file. Replace the URL placeholders:

```
git remote add origin https://github.com/YOUR_USER/YOUR_REPO.git
git remote -v
git push -u origin main
```

To work on an *existing* project instead, run `git clone URL` in its parent directory. Do not initialise inside an existing clone. A ZIP download supplies files but no Git history.

7 Branches and pull requests

From a clean working tree in your practice repository:

```
git switch -c improve-method
```

Edit the README, inspect the diff, then:

```
git add README.md
git commit -m "Explain how the period is measured"
git push -u origin improve-method
```

On GitHub open a pull request into `main`. Explain purpose, changes, and checks. Ask a partner to review; after merging:

```
git switch main
git pull --ff-only
```

Without a remote, practise locally using `git switch main` and `git merge improve-method` instead. Do not blindly combine both routes. Update before beginning new shared work. Commit or deliberately stash unfinished work before switching context. Never force-push to fix a beginner sync problem.

8 Merge conflict practice

Use only the disposable repository, starting clean:

1. Create `conflict-demo` with `git switch -c conflict-demo`.
2. Change an existing README sentence and commit it.
3. Switch to `main`; change that same sentence differently and commit.
4. Run `git merge conflict-demo`. Read the conflict message.
5. Replace the `<<<<<<<`, `=======`, and `>>>>>>>` block with the intended final text.
6. Review the whole result, run `git add README.md`, then `git commit`.
7. Check `git status` and `git log -oneline -graph -all`.

To cancel the unfinished practice merge, use `git merge -abort`. Beginning clean makes this straightforward.

9 Safe recovery

Accidentally staged: After the first commit, `git restore -staged FILE` unstages without changing the working copy.

Revert commit: Starting clean, `git revert ID` adds an inverse commit; resolve conflicts if needed.

Need old content: Inspect `git show COMMIT_ID:path/to/file` first; copy what you need deliberately.

Stash work: `git stash push -u` includes untracked files, not ignored ones. Later use `git stash apply`, inspect, then drop the intended stash.

`git restore FILE` discards uncommitted edits: it is not the same as unstaging. Hard reset, clean, and force-push can destroy work and are not part of the beginner lab. Git cannot usually recover edits it never recorded.

10 Troubleshooting and privacy

Push rejected: Read the error, check branch/remote, and fetch. Do not immediately force-push.

Branches diverged: Use `git fetch origin`, inspect `git log -oneline -graph -all`, and agree on a merge/rebase policy.

Identity unknown: Configure author name/email, not login credentials.

Authentication fails: Check the credential manager or SSH setup without displaying secrets on the projector.

Secret committed: Revoke/rotate immediately. Deleting it in a later commit does not erase history; follow your organisation's incident procedure.

Keep passwords, tokens, private student details, and confidential research data out of public repositories. Git alone is not an independent backup.

11 Further reading

Read Pro Git, the Git reference, and GitHub Skills.